Adaptive detection of Card Not Present credit card fraud under concept drift

Group 5

Decision. Use one integrated GitHub repository with five implementation owners, short-lived reviewed branches, protected main, automated checks, and reproducible evidence. Every completed work session must end with a push so no project work exists only locally.

Scope. The coding team consists of Sanjay Soralamavu Dev, Hemalasya Annapureddy, Aravindan Chidambaram, Mitansh Maheshwari, and Thanyathorn Limsuvattanaphong. This document assigns boundaries, branches, interfaces, tests, merge order, and final acceptance evidence.

Document purpose

This is the team's implementation operating plan. It explains how to obtain and validate the dataset, build the shared Python project, implement every proposed model and experiment, collaborate through GitHub, and generate auditable results without committing raw data or secrets.

Project definition

The project evaluates whether drift-aware adaptive classifiers preserve binary fraud-detection performance better than frozen static classifiers when feature relationships change in a reproducible Card Not Present transaction stream.

Item	Verified project definition
Dataset	CreditTransAct version 1; synthetic credit-card transactions
Official source	https://data.mendeley.com/datasets/y64bbnm2s3/1
Required file	credit_card_fraud.parquet; approximately 487 MB locally
Source population	15,000,000 synthetic transactions
Target	is_fraud; binary transaction-level label
CNP selector	card_present_cnp in {cnp_online, cnp_moto}
Strict CNP population	3,843,653 rows; 564,272 fraud rows; 14.6806% fraud rate
Warm-up	First 500,000 deterministic CNP rows
Evaluation	Remaining 3,343,653 CNP rows
Stream claim	Reproducible pseudo-stream; not authentic documented transaction chronology

Research question

Do ADWIN, DDM, detector-free incremental learning, and an online ensemble retain fraud-detection performance better than frozen Logistic Regression and Random Forest under controlled concept drift and delayed labels?

Claims boundary

- CNP is the transaction channel selected through card_present_cnp; it is not the fraud label.
- is_fraud is the only ground-truth prediction label used by the project.
- The released schema does not provide ground-truth stolen-credential, bot/card-testing, account-takeover, geo-velocity, or open-category classes.
- The project must report binary fraud within CNP transactions and must not claim subtype classification.
- The controlled experiment demonstrates comparative behavior, not production banking readiness.

Required end to end behavior

1. Audit the complete Parquet file, checksum, schema, row count, nulls, channel values, and fraud rates.
2. Filter strict CNP transactions and separate is_fraud before any model input is constructed.
3. Remove direct identifiers and fit imputation, scaling, and categorical encoding only on the warm-up window.
4. Train frozen Logistic Regression and Random Forest baselines on the same warm-up rows.
5. Initialize detector-free incremental learning, ADWIN, DDM, and online bagging models.
6. Deliver deterministic batches and perform prediction before delayed learning.
7. Withhold labels according to the configured delay and release them in exact source-batch order.
8. Inject gradual, label-preserving controlled drift identically for every model.
9. Record batch metrics, phase metrics, detector events, delays, false alarms, configuration, and reproducibility metadata.
10. Generate compact CSV/JSON evidence, static figures, a replay dashboard, and a final report summary.

Processing flow

Stage	Input	Output or invariant
Audit	Raw Parquet file	Validated checksum, schema, counts, values, and null profile
Scope	One source row group	Strict CNP rows; is_fraud separated as the binary label
Preprocess	Warm-up and later frames	Leakage-safe, schema-stable matrices using warm-up-fitted transforms
Predict	Shared batch matrix	Exactly one probability per model for every transaction
Delay	Predictions and source labels	Only labels due under the configured delay become learnable
Adapt	Due labeled batches and alarms	Incremental updates and detector-triggered recovery
Evaluate	Labels, probabilities, and events	CSV/JSON evidence, figures, comparisons, and replay dashboard

Required experiment matrix

The final solution is not complete after one successful run. The following nine requirements form the experimental definition of done.

Requirement	Implementation	Required evidence
Multiple seeds	Run seeds 543, 1543, and 2543	Seed stability mean and standard deviation
No-drift control	Disable the drift injector	Control metrics and detector false alarms
Detector ablation	Compare adaptive_none, ADWIN, and DDM	Same stream, features, labels, and threshold
Label delays	Run 0, 3, 7, and 14 batches	Delay sensitivity table and plot
Drift intensities	Run 0.30, 0.60, and 0.90	Intensity sensitivity table and plot
Detector quality	Measure source/observed delay and false alarms	detector_quality.csv
Online ensemble	Poisson-weighted incremental learner ensemble	Model tests and comparison row
Replay dashboard	Replay batches, phases, metrics, and alarms	Self-contained HTML dashboard
Claim discipline	Document CNP channel and binary label semantics	README, methodology, and report limitations

Model comparison set

Model	Learning behavior	Purpose
Static Logistic	Warm-up training only	Linear frozen baseline
Static Random Forest	Warm-up training only	Nonlinear frozen baseline
Adaptive None	Incremental delayed updates; no reset detector	Detector ablation
Adaptive ADWIN	Incremental updates with ADWIN-triggered recovery	Adaptive detector model
Adaptive DDM	Incremental updates with DDM-triggered recovery	Adaptive detector model
Online ensemble	Five Poisson-weighted incremental learners	Adaptive ensemble baseline

Repository and local environment setup

Clone and prepare the repository

Each member must clone the shared repository into a separate local folder. Do not share virtual environments, raw data folders, or copied project directories through cloud drives.

```
git clone https://github.com/Sanjaysoralamavu/sentinelflow-fraud-detection.git
cd sentinelflow-fraud-detection
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -e '.[dev]'
```

Required project structure

```
sentinelflow-fraud-detection/
├── .github/workflows/ci.yml
├── configs/{full,test,suite,suite-smoke}.yaml
├── data/raw/                  # ignored; local Parquet only
├── data/processed/            # ignored; generated local data
├── docs/
├── results/{metrics,figures,logs,dashboard}/
├── src/sentinelflow/
│   ├── data/{audit,loader,preprocessing}.py
│   ├── streaming/{label_delay,drift_injector}.py
│   ├── models/{static,adaptive,online_ensemble}.py
│   ├── evaluation/{metrics,detectors,visualization}.py
│   ├── config.py  experiment.py  suite.py  dashboard.py
│   └── cli.py  reporting.py
├── tests/
├── Makefile
├── pyproject.toml
└── README.md
```

Files that must never be committed

- data/raw/credit_card_fraud.parquet or any processed copy of the dataset.
- .venv, caches, local IDE configuration, notebook checkpoints, and macOS metadata.
- API keys, access tokens, credentials, personal paths, or environment secrets.
- Large model files and full generated result folders unless the team explicitly approves Git LFS or an artifact store.

```
# Minimum .gitignore entries
.venv/
data/raw/*
data/processed/*
results/metrics/*
results/figures/*
results/logs/*
__pycache__/
.pytest_cache/
.ruff_cache/
*.joblib
*.pkl
```

No API key is required. All modeling, evaluation, and dashboard generation run locally.

Dataset acquisition and Parquet loading

Download procedure

1. Open https://data.mendeley.com/datasets/y64bbnm2s3/1 in a browser.
2. Download credit_card_fraud.parquet from CreditTransAct version 1. If the site provides Download All, extract the archive locally and select this Parquet file.
3. Create data/raw in the cloned repository and place the file at data/raw/credit_card_fraud.parquet.
4. Do not rename columns, resave the Parquet file, or commit it to Git.
5. Record the local SHA-256 and compare it with the team-recorded checksum before experiments.

```
mkdir -p data/raw data/processed
mv /path/to/download/credit_card_fraud.parquet data/raw/
shasum -a 256 data/raw/credit_card_fraud.parquet
ls -lh data/raw/credit_card_fraud.parquet
```

Leader-reference checkpoint. The current verified file SHA-256 is 4a33623216213beb121cdc85006cf36f336601af7645e73feabe1abdf94e0823. If Mendeley replaces the file, record the new version and checksum rather than silently accepting a mismatch.

Memory-safe Parquet inspection

The source has 15 million rows. Do not call pandas.read_parquet on the entire file on an ordinary laptop. Read metadata first and process row groups incrementally.

```
from pathlib import Path
import pyarrow.parquet as pq

path = Path("data/raw/credit_card_fraud.parquet")
parquet = pq.ParquetFile(path)
print("rows:", parquet.metadata.num_rows)
print("row groups:", parquet.num_row_groups)
print("columns:", parquet.schema.names)

for group_id in range(parquet.num_row_groups):
    frame = parquet.read_row_group(group_id).to_pandas()
    cnp = frame[frame["card_present_cnp"].isin(["cnp_online", "cnp_moto"])]
    labels = cnp["is_fraud"].astype("int8")
    features = cnp.drop(columns=["is_fraud", "customer_id", "transaction_id"])
    # Pass this bounded frame to the audit or stream layer; do not accumulate all rows.
```

Dataset audit acceptance checks

Check	Expected behavior
File presence	Fail clearly if the Parquet path is missing
Source rows	Metadata reports 15,000,000 rows
Required columns	card_present_cnp and is_fraud exist
Target values	Only binary 0/1 values after validation
CNP values	Strict filter retains cnp_online and cnp_moto
CNP population	3,843,653 rows, including 564,272 is_fraud = 1 rows

Five person ownership and branch plan

Each member owns a package boundary and its tests. A branch is not complete merely because code runs locally; completion requires pushed commits, a pull request, passing CI, review approval, and generated evidence.

Member	Primary branch	Owned area	Required reviewer
Sanjay Soralamavu Dev	feat/sanjay-project-integration	Architecture, configuration, static baselines, runner, suite, CI, final integration	Aravindan
Aravindan Chidambaram	feat/aravindan-data-pipeline	Parquet audit, loader, strict CNP filter, preprocessing, leakage controls	Sanjay
Mitansh Maheshwari	feat/mitansh-stream-drift	Pseudo-stream, batches, delayed-label queue, controlled drift, scenario inputs	Hemalasya
Hemalasya Annapureddy	feat/hemalasya-adaptive-models	Adaptive models, ADWIN, DDM, detector-free model, online ensemble, recovery	Mitansh
Thanyathorn Limsuvattanaphong	feat/thanyathorn-evaluation-dashboard	Metrics, detector audit, plots, reports, replay dashboard	Sanjay

Branch creation commands

```
git switch main
git pull --ff-only origin main
git switch -c feat/<assigned-branch-name>
git push -u origin feat/<assigned-branch-name>
```

The assigned branch names are the first implementation branches. After a branch is merged, follow-up work must start from current main on a new focused branch such as fix/hemalasya-ddm-reset or feat/thanyathorn-delay-plot. Do not keep long-lived personal branches.

Shared interface freeze

Mitansh and Hemalasya own the most difficult and interdependent components. Before feature implementation, they must jointly document the batch record, delayed-label record, prediction array, detector input, drift event, and update order. Interface changes require a linked issue and approval from both owners.

Contract	Required fields or behavior	Owner
Batch	batch_id, source rows, progress, phase, feature frame, labels	Mitansh
Delayed labels	source_batch, observed_batch, features, labels, predictions	Mitansh
Model prediction	One fraud probability per row; no learning before scoring	Hemalasya
Drift event	detector, source batch, observed batch, phase, alarm type	Hemalasya
Metrics record	model, batch, confusion counts, precision, recall, F1, FPR, AUC values	Thanyathorn
Configuration	seed, delay, drift toggle/intensity, paths, model parameters	Sanjay

Contract change procedure

- Open a GitHub issue describing the old contract, proposed contract, affected owners, and migration plan.
- Add or update a contract test before changing production code.
- Obtain approval from every owner whose package consumes the changed interface.
- Merge the contract change before dependent feature changes and update the integration branch from main.

Work package 1 Sanjay project integration

Primary branch. feat/sanjay-project-integration

Owned paths. src/sentinelflow/config.py, models/static.py, experiment.py, suite.py, cli.py, configs, Makefile, CI workflow, and integration documentation.

Implementation tasks

1. Create the package skeleton, editable installation, Makefile targets, configuration loader, and CLI commands.
2. Define one YAML configuration contract for paths, warm-up size, batch size, seed, label delay, drift schedule, detector parameters, output directories, and model settings.
3. Implement frozen Logistic Regression and Random Forest baselines trained only on warm-up rows.
4. Build the end-to-end experiment coordinator without duplicating data, stream, adaptive-model, or evaluation logic owned by others.
5. Build the nine-scenario suite orchestrator with deduplication so identical baseline scenarios are not rerun accidentally.
6. Configure GitHub Actions to install the package, run Ruff, and run Pytest for every pull request.
7. Integrate branches in the approved order, resolve contract-level conflicts with owners, and tag the final reproducible revision.

Acceptance criteria

- Static models do not update after warm-up.
- Every run writes resolved configuration, seed, dataset checksum, row counts, and elapsed time.
- Smoke and full commands use the same code path; only configuration and row limits differ.
- A clean checkout passes make lint and make test before merge.
- No teammate must edit experiment.py merely to add their component; integrations occur through agreed interfaces.

Required pull request evidence

- CLI help output and example smoke command.
- Static model unit tests and freeze assertion.
- Suite manifest with the nine unique scenarios.
- Passing CI link and integration notes.

Work package 2 Aravindan data and preprocessing

Primary branch. feat/aravindan-data-pipeline

Owned paths. src/sentinelflow/data/audit.py, loader.py, preprocessing.py, tests/test_loader.py, and dataset setup documentation.

Implementation tasks

1. Implement a Parquet metadata audit that does not load all 15 million rows into memory.
2. Validate required columns, binary target values, channel values, row counts, missing values, fraud counts, and SHA-256.
3. Yield bounded strict-CNP batches by filtering cnp_online and cnp_moto while iterating Parquet row groups.
4. Separate is_fraud before preprocessing and drop customer_id and transaction_id from model features.
5. Build a ColumnTransformer with numeric imputation/scaling and categorical imputation/one-hot encoding with unknown-category handling.
6. Fit the preprocessor only on the 500,000-row warm-up and transform every later batch without refitting.
7. Expose a feature manifest containing included, dropped, numeric, and categorical columns.

Acceptance criteria

- The full verified dataset produces exactly 3,843,653 strict-CNP rows and 564,272 fraud rows.
- Peak memory remains bounded because row groups are processed incrementally.
- The transformed feature matrix contains no target or identifier leakage.
- Repeated loading with the same seed produces the same batch hashes.
- Missing file, missing columns, nonbinary target, and unknown channel values receive explicit errors or audit warnings.

Required pull request evidence

- dataset_audit.json
- channel_summary.csv
- feature manifest
- loader and leakage tests
- documented download and checksum procedure

Work package 3 Mitansh stream and controlled drift

Primary branch. feat/mitansh-stream-drift

Owned paths. src/sentinelflow/streaming/label_delay.py, drift_injector.py, stream contracts, and tests/test_streaming.py.

Implementation tasks

1. Create deterministic batch IDs and reproducible within-batch ordering using the configured seed.
2. Implement prequential order: receive batch, inject configured drift, preprocess, predict all models, record metrics, enqueue labels, then learn only from labels that become due.
3. Implement a delayed-label queue supporting 0, 3, 7, and 14 batch delays and flushing due records at stream end.
4. Implement a no-drift control in which every feature remains unchanged and any detector alarm is classified as false.
5. Implement gradual drift beginning at 45% progress, reaching full configured intensity at 70%, and remaining drifted thereafter.
6. Apply drift only to selected fraud-row features, keep labels unchanged, and log affected counts, phase, and intensity.
7. Support maximum intensities 0.30, 0.60, and 0.90 without changing the model interface.

Acceptance criteria

- Two runs with the same seed generate identical phases, affected-row counts, and batch hashes.
- The no-drift configuration changes zero feature values.
- Controlled drift never changes is_fraud.
- Observed release batch equals source batch plus configured delay.
- Delay zero still follows prediction-before-learning and does not leak current labels into current predictions.

Required pull request evidence

- label-delay boundary tests
- label-preservation assertion
- no-drift test
- drift-phase log
- reproducibility test across repeated seeds

Work package 4 Hemalasya adaptive learning

Primary branch. feat/hemalasya-adaptive-models

Owned paths. src/sentinelflow/models/adaptive.py, online_ensemble.py, common.py, and adaptive sections of tests/test_models.py.

Implementation tasks

1. Implement one detector-free incremental classifier that learns from delayed labels but never resets.
2. Implement ADWIN and DDM variants receiving the same delayed prediction-error stream.
3. Record detector name, source batch, observed batch, alarm state, reset count, and recovery metadata.
4. On confirmed alarms, initialize a fresh incremental learner and retrain it using only the bounded recent labeled window.
5. Ensure no future or unreleased label is available to the model or detector.
6. Implement an online bagging ensemble with five independent incremental learners, Poisson(1) sample weights, and mean fraud probability.
7. Use class weighting or bounded imbalance handling consistently and document all thresholds and reset parameters.

Acceptance criteria

- All adaptive variants receive identical due batches and emit one probability per row.
- Detector-free learning performs zero detector-triggered resets.
- ADWIN and DDM alarms reproduce under a fixed seed and configuration.
- Reset retraining uses only already released labels from the bounded recent window.
- The online ensemble has independent learners and reproducible Poisson weights.
- Unit tests verify delayed learning, resets, probabilities, and ensemble updates.

Required pull request evidence

- model behavior tests
- alarm and reset log
- adaptive comparison on the smoke stream
- documented detector parameters
- online ensemble test

Work package 5 Thanyathorn evaluation and dashboard

Primary branch. feat/thanyathorn-evaluation-dashboard

Owned paths. src/sentinelflow/evaluation, dashboard.py, reporting.py, evaluation tests, and result-interpretation documentation.

Implementation tasks

1. Calculate confusion counts, precision, recall, F1, false-positive rate, ROC-AUC, and PR-AUC for each model on identical rows.
2. Produce per-batch, per-phase, and overall summaries without averaging incompatible sample sizes.
3. Define the first nonzero injected-drift batch as ground-truth drift start for controlled experiments.
4. Classify alarms before drift, and every alarm in no-drift controls, as false alarms.
5. Report source detection delay excluding label latency and observed delay including label latency.
6. Generate model comparison, drift timeline, label-delay sensitivity, drift-intensity sensitivity, and seed-stability outputs.
7. Build a self-contained replay dashboard with play/pause, speed, model selection, batch slider, phases, alarms, and changing metrics.
8. Generate report text that preserves dataset and claims limitations.

Acceptance criteria

- Metric calculations match hand-computed small examples.
- All models are evaluated on exactly the same rows and threshold policy.
- Detector tables include total alarms, false alarms, post-drift alarms, missed detection, and both delay definitions.
- Plots label axes, models, phases, and alarm types clearly.
- The dashboard opens locally without a server or API key.
- Every reported number identifies its generating CSV or JSON artifact.

Required pull request evidence

- metric unit tests
- detector-quality tests
- sample figures
- dashboard screenshot or HTML
- CSV-to-report traceability checklist

GitHub delivery workflow

1. Leader creates one issue per deliverable with owner, reviewer, dependency, acceptance criteria, and expected evidence.
2. Owner updates local main and creates the assigned short-lived branch.
3. Owner opens a draft pull request after the first meaningful commit and links the issue.
4. Owner commits implementation, tests, configuration, and documentation in small reviewable units.
5. Owner pushes at the end of every work session. Completed changes may never exist only locally.
6. Owner runs lint, unit tests, and the relevant smoke test before requesting review.
7. Reviewer checks contracts, tests, evidence, and scope; comments must be resolved in the same pull request.
8. Leader merges only after CI passes and required review approval is recorded.
9. Merged remote branches are deleted; new work starts from refreshed main.

Daily Git commands

```
git status
git add <specific-files>
git commit -m "feat: describe one completed unit"
git push

# Before starting follow-up work after a merge
git switch main
git pull --ff-only origin main
git switch -c feat/<owner>-<focused-task>
```

Pull request description must include

- Linked issue and concise scope statement.
- Files and interfaces changed.
- Tests added and exact commands executed.
- Evidence files or screenshots generated.
- Known limitations and follow-up work.
- Confirmation that raw data, credentials, local paths, and large artifacts are absent.

Merge and conflict rules

- Never force-push main or merge with failing CI.
- Do not resolve another owner's functional conflict without that owner's review.
- Do not modify shared interfaces silently; update the contract issue first.
- Prefer focused pull requests under approximately 400 changed lines when practical.
- Use squash merge after approvals so main keeps one traceable commit per issue.

Integration order and dependency gates

Gate	Merge owner	Dependency	Leader acceptance
1 Repository skeleton	Sanjay	None	CI, configuration schema, package install
2 Data and preprocessing	Aravindan	Gate 1	Checksum, counts, leakage and memory tests
3 Stream and delay	Mitansh	Gates 1-2	Reproducible batches, delay and label preservation
4 Static baselines	Sanjay	Gates 1-2	Frozen-after-warm-up assertion
5 Adaptive models	Hemalasya	Gates 1-3	Delayed learning, alarms, resets, ensemble tests
6 Evaluation	Thanyathorn	Gates 3-5	Metric and detector-quality tests
7 End-to-end smoke	Sanjay	Gates 1-6	All outputs from bounded evaluation
8 Full suite	All; Sanjay runs	Gate 7	Nine scenarios, stable manifests and figures
9 Final audit	All	Gate 8	Claims and results trace to evidence

Recommended execution cadence

Iteration	Parallel work	Exit condition
Iteration 1	Sanjay skeleton; Aravindan audit; Mitansh/Hemalasya contract design	Repository installs and core contracts approved
Iteration 2	Data pipeline; stream engine; static baselines; metric prototypes	Unit and contract tests pass
Iteration 3	Adaptive models; detectors; ensemble; controlled drift	End-to-end smoke run succeeds
Iteration 4	Nine-scenario suite; dashboard; sensitivity plots	Full outputs and numerical audit complete
Iteration 5	Reproduction, documentation, demonstration, final fixes	Clean checkout and submission tag approved

Leader monitoring routine

- Review open issues, draft pull requests, CI, and blockers at least twice each week.
- Use GitHub Projects or an issue board with Backlog, In Progress, Review, Blocked, and Done columns.
- Reject completion claims without tests and generated evidence.
- Require a second owner when a critical dependency is blocked for more than one work session.
- Freeze full-run configuration before evidence generation; later changes require an issue and rerun decision.

Testing and execution commands

Command	Purpose	When required
make install	Create/install development environment	First setup or dependency change
make audit	Validate full dataset metadata and CNP counts	Data pipeline PR and final run
make lint	Run Ruff checks	Before every PR review
make test	Run all Pytest tests	Before every PR review
make run-suite-smoke	Run all scenarios on bounded rows	Before integration merge
make run-full	Run the main full experiment	After smoke passes
make run-suite	Run full nine-scenario experiment matrix	Final evidence generation
make dashboard	Regenerate replay dashboard	After final full run

Test levels

Level	Examples	Pass rule
Unit	CNP filter, queue delay, model reset, metric calculation	Normal and boundary cases pass
Contract	Batch schema, feature dimensions, event schema	Owners agree on unchanged interface
Smoke	Nine scenarios with bounded evaluation rows	All expected artifacts generated
Full	3,343,653 evaluation CNP rows	No crash; outputs complete
Reproduction	Fresh clone and local dataset	Independent member follows README successfully
Report audit	Tables, plots, detector statements	Every claim traces to CSV or JSON

Pull request definition of done

- Issue acceptance criteria are satisfied without unrelated changes.
- New behavior has tests and existing tests still pass.
- Configuration and documentation are updated with the code.
- Fixed seeds and reproduction commands are recorded.
- No raw dataset, secret, personal path, cache, or large local artifact is tracked.
- Required reviewer approves and GitHub Actions passes.

Expected outputs and audit checkpoints

Output	Responsible owner	Minimum content
dataset_audit.json	Aravindan	Checksum, schema, rows, channels, fraud counts, nulls
model_summary.csv	Thanyathorn	All six models and overall metrics
batch_metrics.csv	Thanyathorn	Per-batch metrics for replay and plots
drift_events.csv	Hemalasya and Thanyathorn	Alarm source/observed batch and classification
detector_quality.csv	Thanyathorn	Delays, false alarms, missed detection
suite_manifest.csv	Sanjay	Scenario settings, seed, delay, intensity, status
seed_stability.csv	Thanyathorn	Mean and standard deviation across seeds
Sensitivity figures	Thanyathorn	Delay and intensity comparisons
dashboard/index.html	Thanyathorn	Self-contained replay dashboard
Run metadata	Sanjay	Resolved config, checksum, versions, timing

Leader reference checkpoints

These values are audit references for the current full configuration, not scores the team should hard-code. Differences are acceptable only when caused by documented design or configuration changes.

Model	Precision	Recall	F1	FPR	PR AUC
Adaptive ADWIN	0.9969	0.9938	0.9953	0.00054	0.9977
Adaptive DDM	0.9967	0.9944	0.9955	0.00058	0.9972
Adaptive None	0.9975	0.9577	0.9772	0.00042	0.9975
Online ensemble	0.9976	0.9539	0.9753	0.00040	0.9965
Static Random Forest	0.9996	0.9661	0.9826	0.00007	0.9959
Static Logistic	0.9966	0.8980	0.9447	0.00053	0.9951

Detector checkpoint. In the main strong-drift run, controlled drift starts at batch 190. ADWIN reports zero false alarms and a source delay of 7 batches; DDM reports three false alarms and a source delay of 9 batches. Observed delays include the configured label delay.

Final submission package

Deliverable	Owner	Final verification
Integrated source and configuration	Sanjay	Clean checkout passes installation, lint, tests, and smoke suite
Dataset instructions and checksum	Aravindan	Fresh download and audit reproduce verified counts
Stream, delay, and drift evidence	Mitansh	Configuration and logs prove label preservation and timing
Adaptive and ensemble evidence	Hemalasya	Tests and alarms prove delayed learning and resets
Tables, figures, detector audit, dashboard	Thanyathorn	Numbers match exported evidence
Integrated demonstration	All five; led by Sanjay	Five-minute walkthrough uses saved final outputs

Final go or no go checklist

- Project title and scope consistently identify adaptive binary CNP fraud detection under controlled concept drift.
- CNP is described as a channel and is_fraud as the binary target.
- No unsupported stolen-credential, velocity, account-takeover, bot-testing, open-category, or natural-drift claim remains.
- All nine required experimental improvements are present in configuration, code, tests, and outputs.
- The no-drift control and detector-free ablation are included in final results.
- All tests and CI pass on the tagged submission revision.
- Every table, plot, and conclusion is traceable to generated evidence.
- Raw data, secrets, personal paths, and local environments are absent from Git history.
- All five implementation owners have visible issues, commits, pull requests, reviews, and merge evidence.

Leader approval record

Gate	Owner	Reviewer	Status or evidence link
Repository and CI	Sanjay	Aravindan	
Dataset and preprocessing	Aravindan	Sanjay	
Stream, delay, and drift	Mitansh	Hemalasya	
Adaptive models and ensemble	Hemalasya	Mitansh	
Evaluation and dashboard	Thanyathorn	Sanjay	
Full-suite reproduction	Sanjay	All five	

Reference locations

- Implementation overview: README.md
- Configuration: configs/full.yaml and configs/suite.yaml
- Methodology: docs/METHODOLOGY.md
- Results interpretation: docs/RESULTS_INTERPRETATION.md
- Demonstration procedure: docs/DEMO_GUIDE.md
- Suite evidence: results/suite/SUITE_SUMMARY.md
- Replay dashboard: results/dashboard/index.html

Immediate next steps

1. Confirm every member has accepted the GitHub invitation and can clone, branch, commit, and push.
2. Create the five implementation issues using the work-package acceptance criteria in this guide.
3. Have every member download the Parquet file locally and verify the recorded SHA-256.
4. Merge the repository skeleton and interface-contract pull request before component development diverges.
5. Schedule the first checkpoint after unit and contract tests pass, before any full dataset run.

Team rule

A task is not complete until its code, tests, configuration, documentation, and evidence are pushed, reviewed, and merged. Local-only work does not count as project progress.